Recognition of Emergency Vehicle Sirens and Velocity Estimation

Advanced Driver Assistance Systems Project

Bersellini Andrea

Domeniconi Giacomo

Sound Detection

How can the detection of environmental sounds be important in automotive applications?

- Supporting other ADAS for environmental awareness.
- Detecting specifics sounds (**emergency vehicles**, alarms, voices, ...) to improve safety.
- Increase amount of data for data analysis.

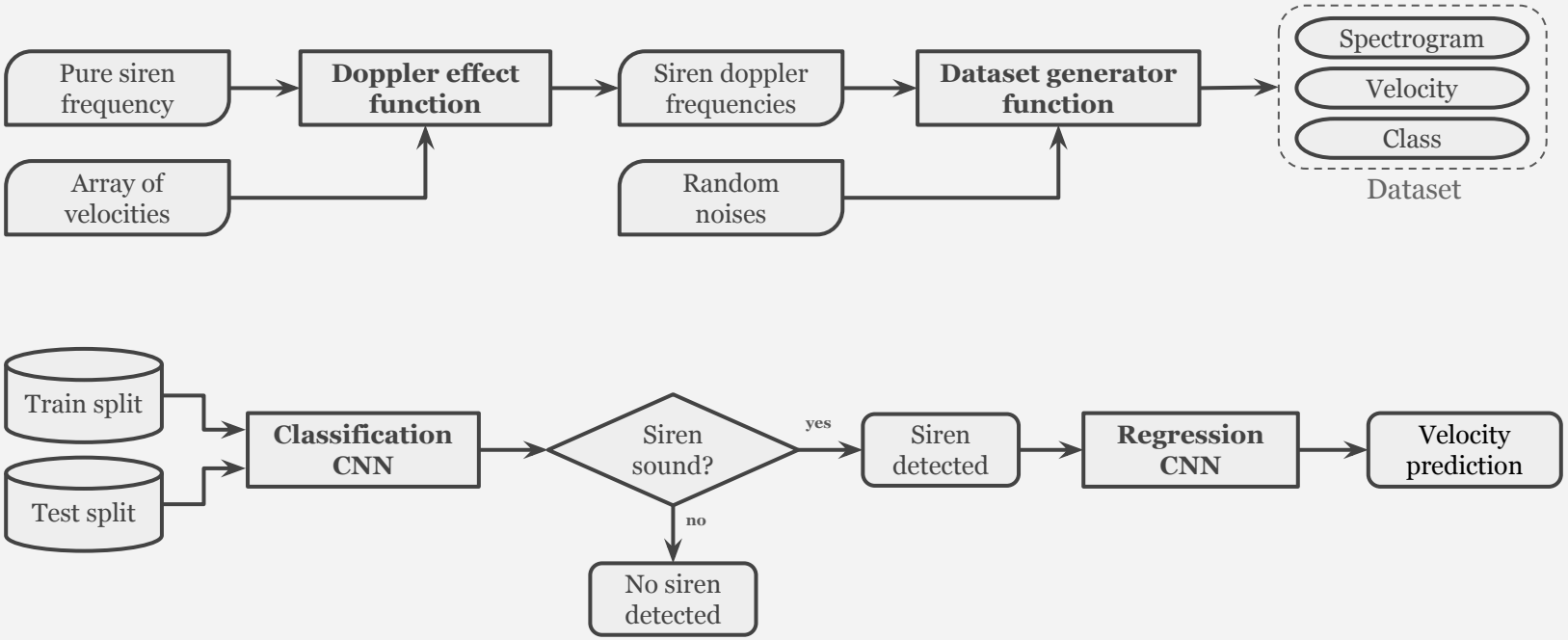
State of the Art

- Microphones: they can only perceive sounds and direction of the source.
- Dataset problem: gathering emergency siren data can be difficult and problematic, so there are virtually no usable (and free) data.
- Deep Learning for sound detection: classification to identify a specific sound and regression to estimate a specific value.
- Spectrograms are the most used images for sound classification in CNNs.

Requirements

- Sensors: microphones to gather the audio data.
- Hardware integration (mounting, calibration, ...) and known location of each microphone on the car for the best efficiency.
- Complete audio data for the training. The dataset must contain realistic data, binary classes to identify sirens and velocity of the emergency vehicle (positive = distancing, negative = approaching).
- Classification for sound recognition.
- Regression for velocity estimation.
- Practical testing in a real life environment.

Architecture

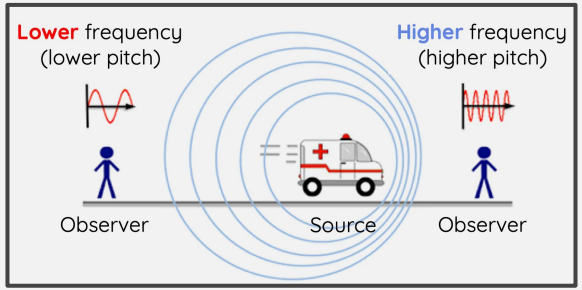


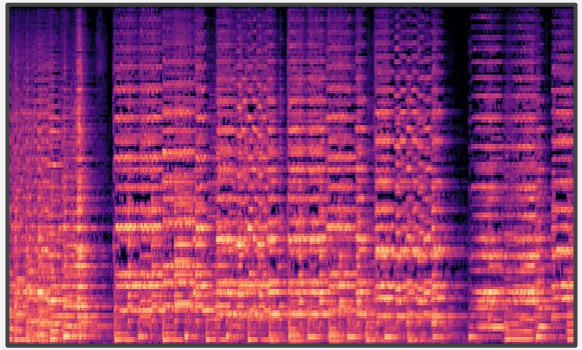
Dataset Generation

- **Doppler effect**: $f_o = \frac{v + v_o}{v + v_s} f_s$

- **Mel-Spectrogram**: Time-frequency visual representation of a sound in a Mel-scale

- **Noise classes**: rain, voices, engines, animals, horns, brakes, constructions, ecc…



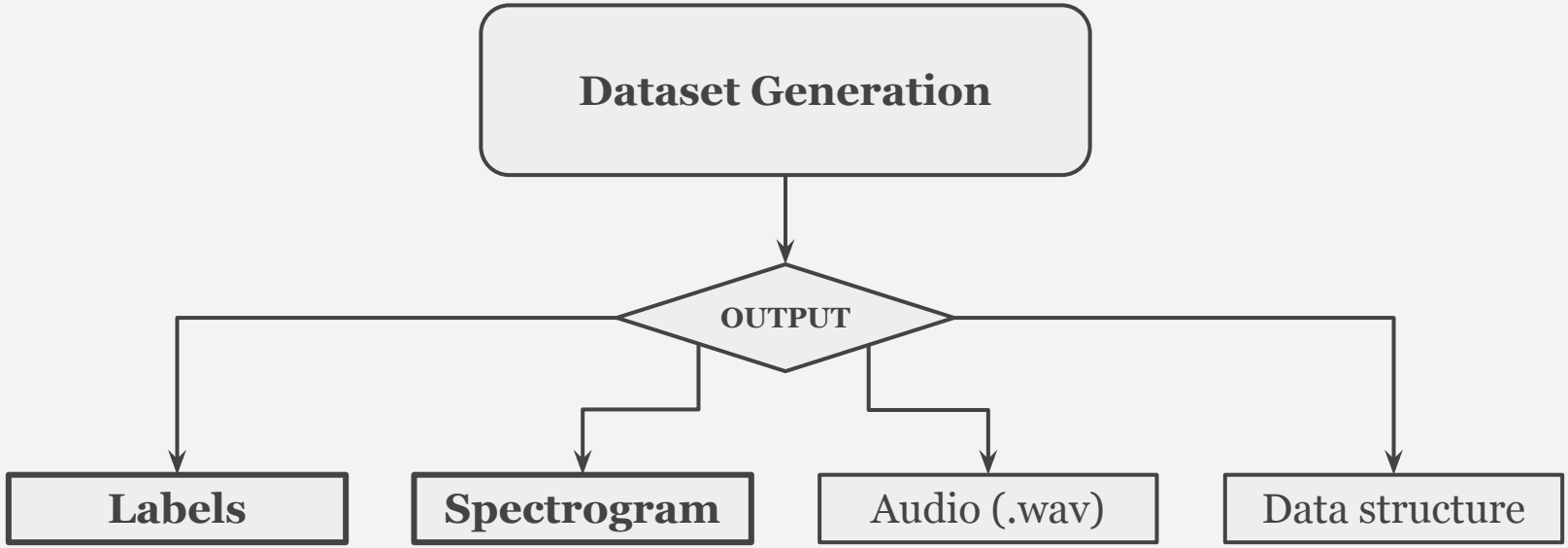


Dataset Generation

Specific data needs → Generation of artificial dataset… How?

- Starting from "pure" siren frequency.
- Simulating movement through Doppler effect
- Adding random noise to the moving siren to simulate a real life urban environment.
- Shifting and cutting the track to improve diversity and standardizing the dimensions.
- Converting the audio files to Mel spectrograms.
- Repeating the process to obtain a large number of combinations.

Output Format

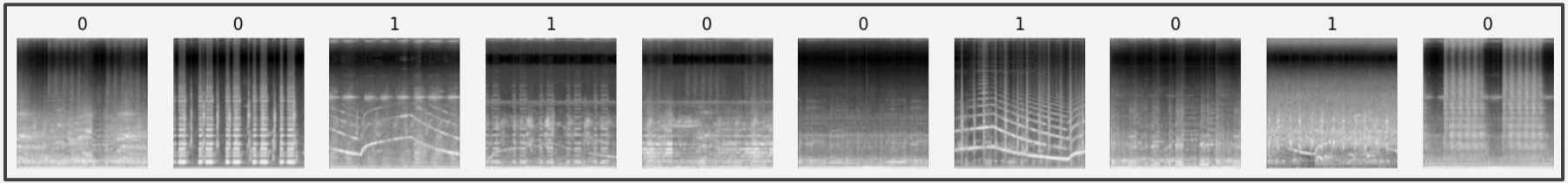


Classification

- Mel spectrogram
- Velocity (positive or negative)

CNN

Predicted class (0,1)



Classification

Hyperparameters

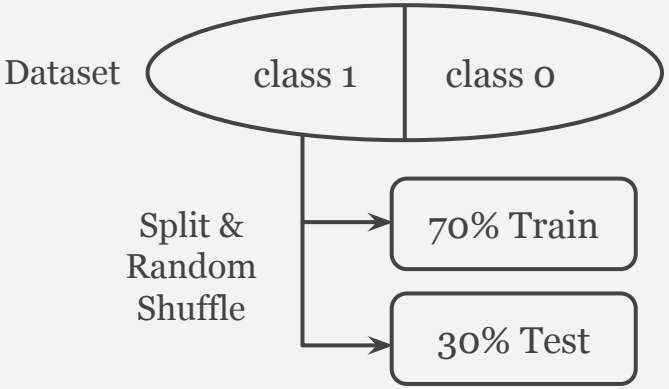
Images : 128x128 (grayscale)

Batch size : 32

Epochs : 20

Learning Rate : 0.001

Loss function : BCE



$$\text{ACCURACY} = \frac{TP + TN}{TP + TN + FP + FN}$$

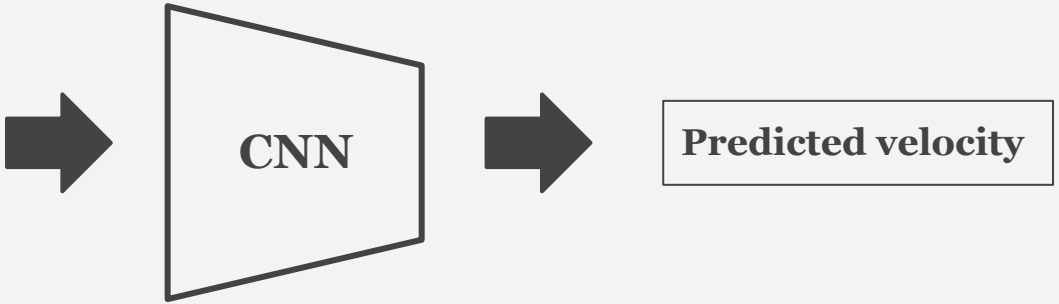
Code

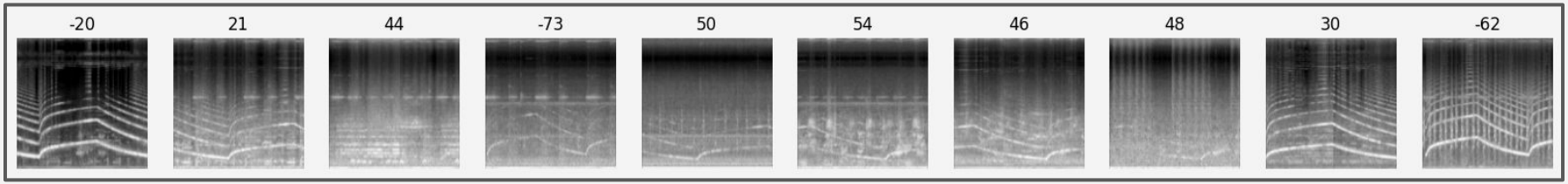
```
self.conv1 = nn.Conv2d(1, 16, 3, 1, 1)
self.conv2 = nn.Conv2d(16, 32, 3, 1, 1)
self.conv3 = nn.Conv2d(32, 64, 3, 1, 1)

self.bn1 = nn.BatchNorm2d(16)
self.bn2 = nn.BatchNorm2d(32)
self.bn3 = nn.BatchNorm2d(64)

self.fc1 = nn.Linear(128 * 128, 512)
self.fc2 = nn.Linear(512, 1)

self.pool = nn.MaxPool2d(2, 2, 0)
self.dropout = nn.Dropout(0.5)
```

```
def forward(self, x):
    x = self.pool(F.relu(self.bn1(self.conv1(x))))
    x = self.pool(F.relu(self.bn2(self.conv2(x))))
    x = self.pool(F.relu(self.bn3(self.conv3(x))))

    x = torch.flatten(x, start_dim=1)

    x = F.relu(self.fc1(x))
    x = self.dropout(x)

    x = torch.sigmoid(self.fc2(x))
    return x
```

Regression

- Mel spectrogram
- Velocity (positive or negative)





Regression

Hyperparameters

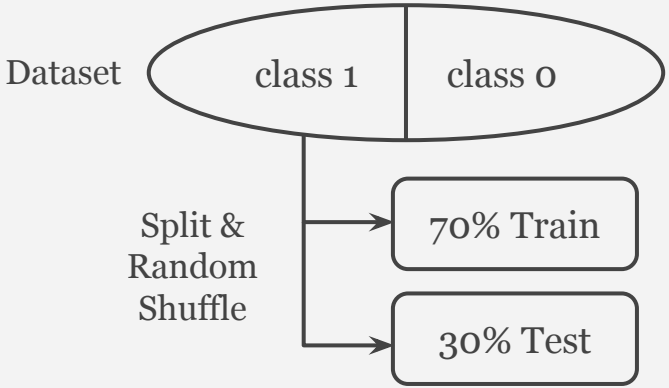
Images : 128x128 (grayscale)

Batch size : 32

Epochs : 20

Learning Rate : 0.001

Loss function : MSE



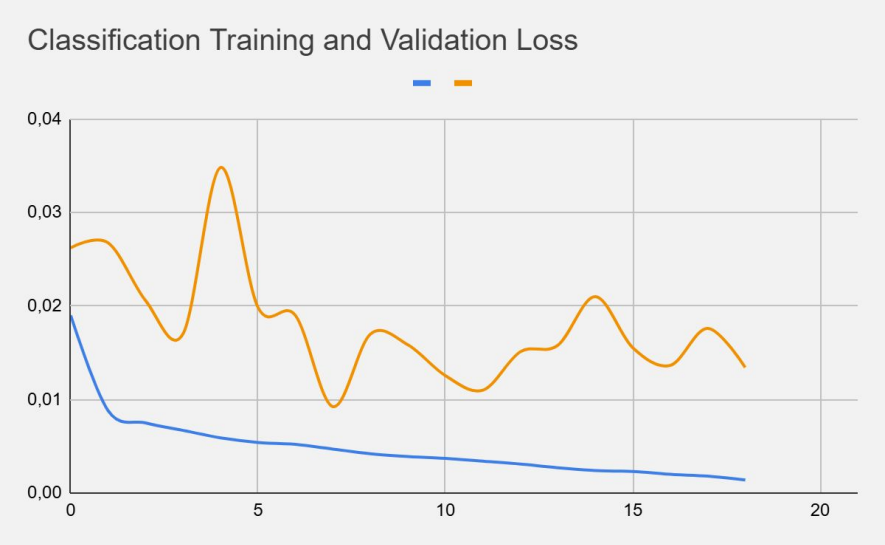
$$\text{SMAPE} = \frac{1}{n}\sum_{t=1}^{n}\frac{|F_t - A_t|}{(A_t + F_t)/2}$$

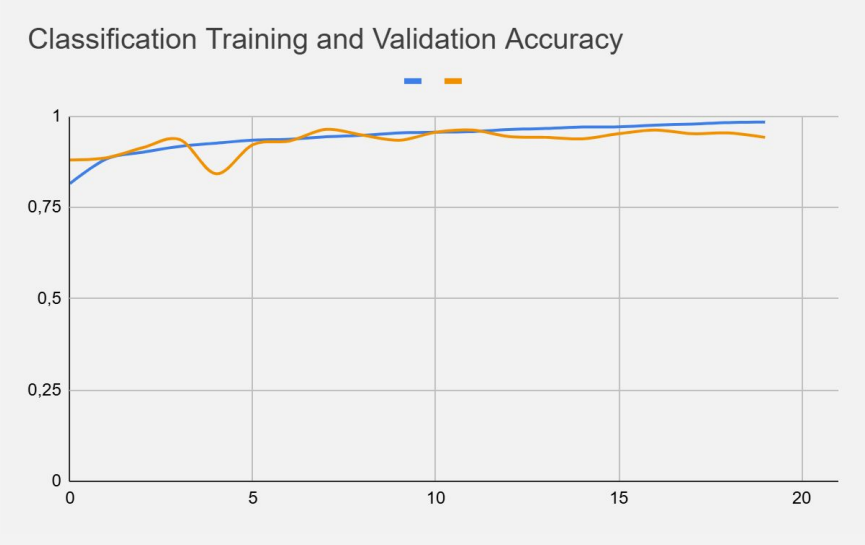
Code

```python
self.conv1 = nn.Conv2d(1, 16, 3, 1, 1)
self.conv2 = nn.Conv2d(16, 32, 3, 1, 1)
self.conv3 = nn.Conv2d(32, 64, 3, 1, 1)

self.bn1 = nn.BatchNorm2d(16)
self.bn2 = nn.BatchNorm2d(32)
self.bn3 = nn.BatchNorm2d(64)

self.fc1 = nn.Linear(128 * 128, 512)
self.fc2 = nn.Linear(512, 1)

self.pool = nn.MaxPool2d(2, 2, 0)
self.dropout = nn.Dropout(0.5)
```

```python
def forward(self, x):
    x = self.pool(F.relu(self.bn1(self.conv1(x))))
    x = self.pool(F.relu(self.bn2(self.conv2(x))))
    x = self.pool(F.relu(self.bn3(self.conv3(x))))

    x = torch.flatten(x, start_dim=1)

    x = F.relu(self.fc1(x))
    x = self.dropout(x)

    x = self.fc2(x) # NO SIGMOID
    return x
```

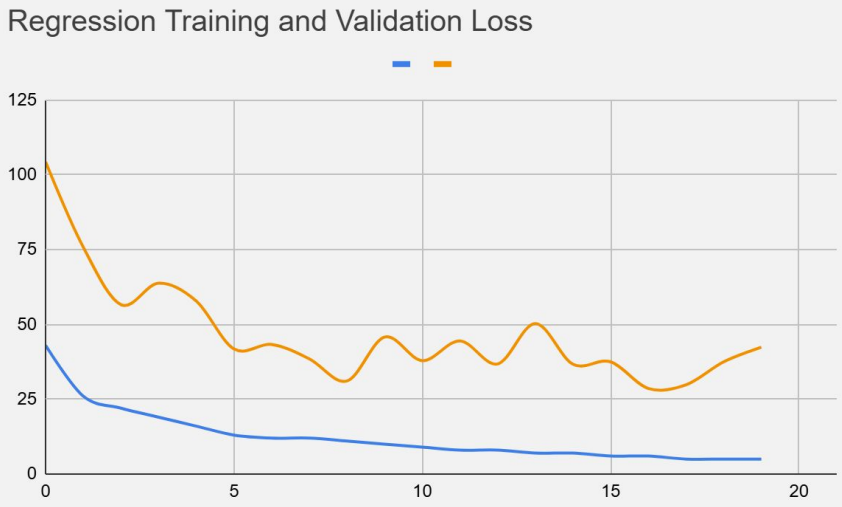
Results

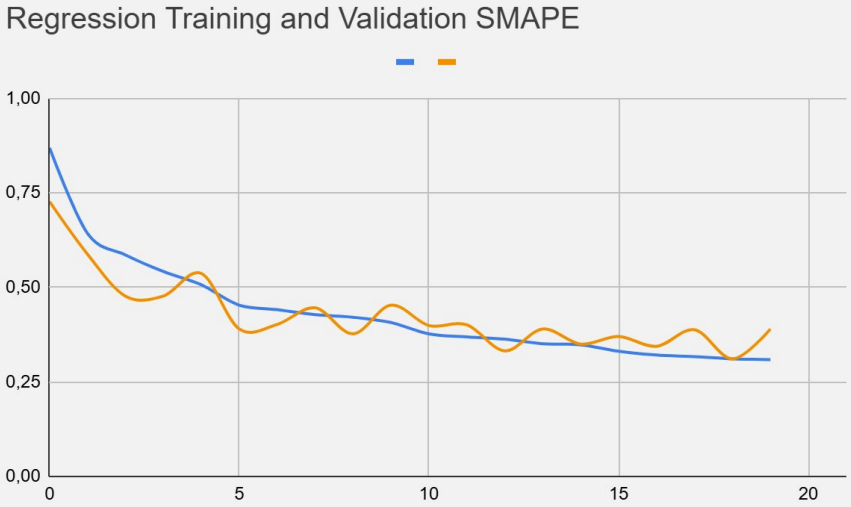




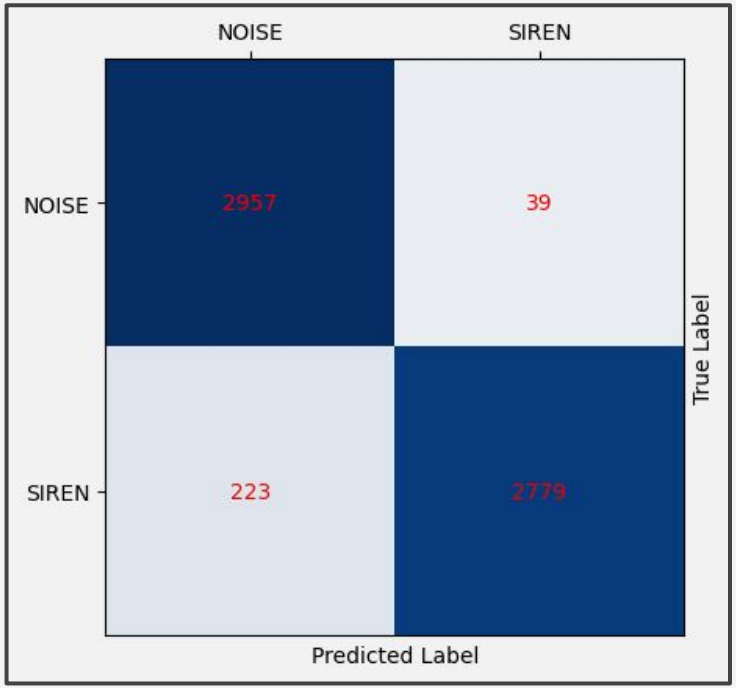
Variance in the validation metrics caused by small sample size (only 500 images/epoch to reduce training times)

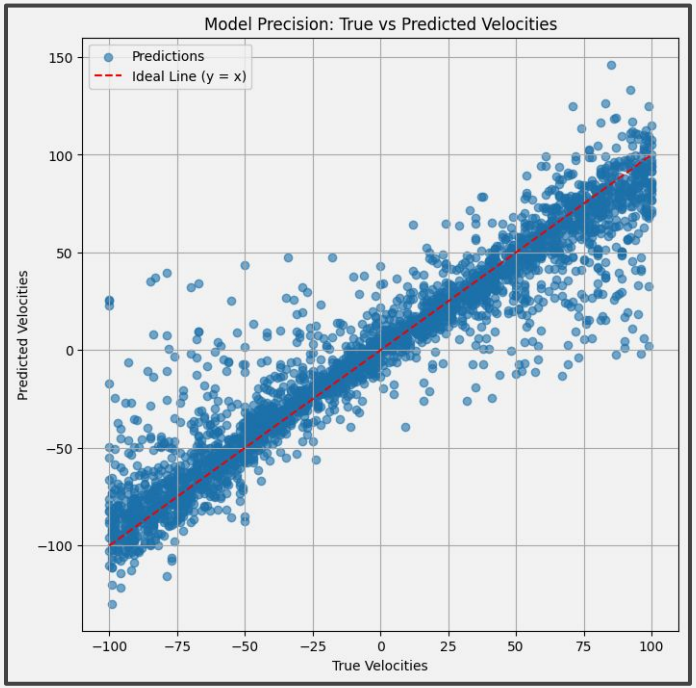
Results


Regression Training and Validation Loss
125
100
75
50
25
0
0
5
10
15
20


Regression Training and Validation SMAPE
1,00
0,75
0,50
0,25
0,00
0
5
10
15
20

Results


NOISE
SIREN
NOISE
SIREN
2957
39
223
2779
True Label
Predicted Label


Model Precision: True vs Predicted Velocities
Predictions
Ideal Line (y = x)
150
100
50
0
−50
−100
Predicted Velocities
−100
−75
−50
−25
0
25
50
75
100
True Velocities

Assumptions and Improvements

- Assumption of moving source and static receiver (can be solved by adding doppler to every sound).
- In a realistic scenario, building, weather and unpredictable events, can obstruct the sound wave and give false measurements.
- Adding more noises and different types of sirens to achieve better results in a real scenario.
- Array of microphones for sound direction estimation: (DeepEar).

Conclusions

Dataset:

- Faster and easier compare to manual acquisition
- Possibility of generating every possible sound combination
- Generate data using multiple formats

Networks:

- The network allowed to identify features using a Mel spectrogram of the desired sound
- Acceptable performances for developing phase, but not enough for real life applications

Thanks for the attention!

References

- DeepEar: Sound Localization With Binaural Microphones,Qiang Yang , Student Member, IEEE and Yuanqing Zheng https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9954178.

- Sound Classification Using Convolutional Neural Networks, Kaustumbh Jaiswal, Dhairya Kalpeshbhai Pate https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8648629

- Acoustic Localization of Emergency Vehicle Sirens in Traffic, Mattes Ohlenbusch1,2, Christian Rollwage1, Joerg Bitzer https://pub.dega-akustik.de/DAGA_2020/data/articles/000010.pdf